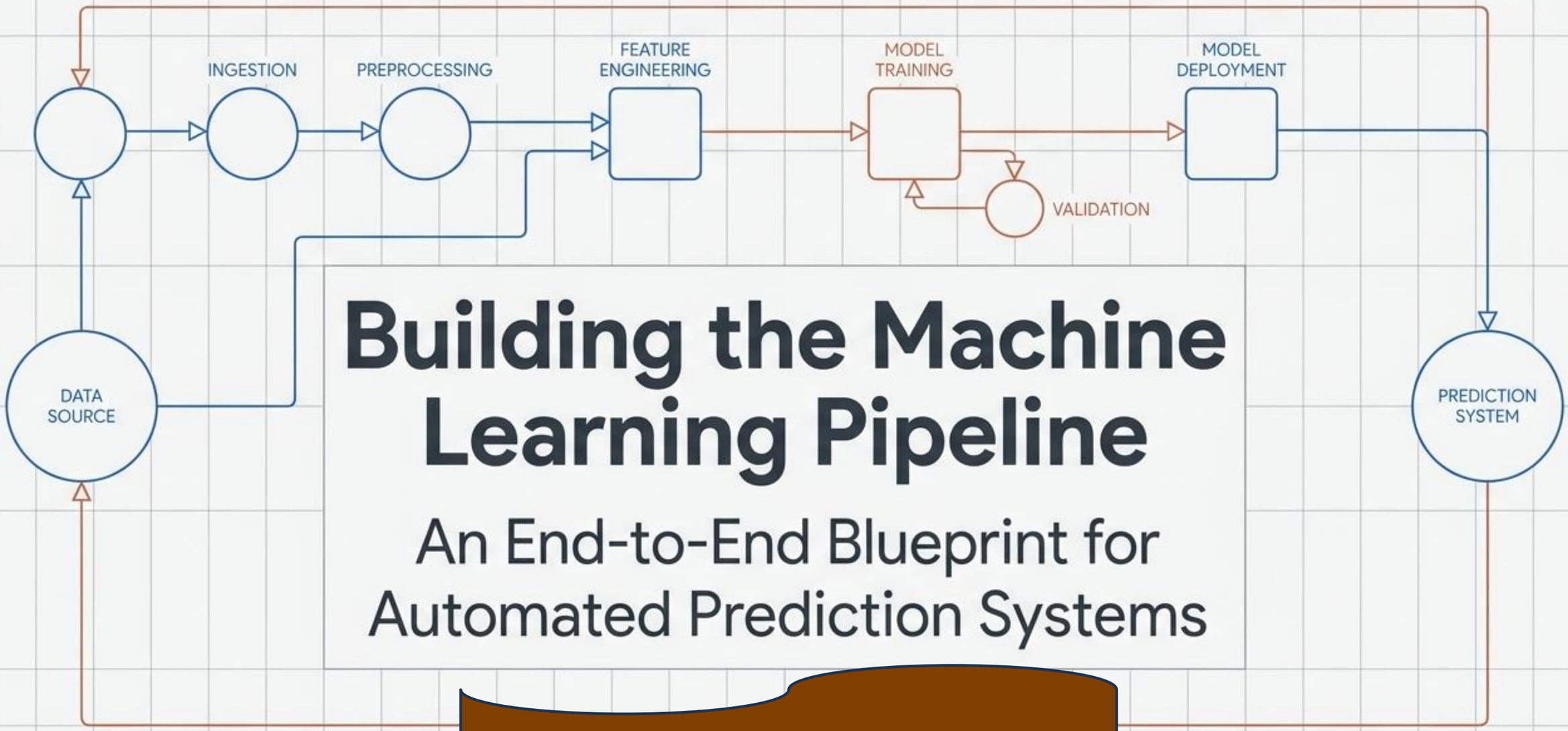


Building the Machine Learning Pipeline

An End-to-End Blueprint for Automated Prediction Systems

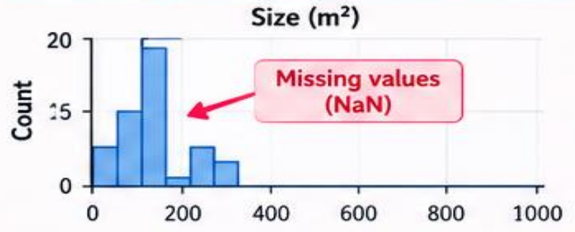
Prof. Dr. Mohammed Elmogy
Dr. Dina Saif

Example Dataset with Many Common Data Problems

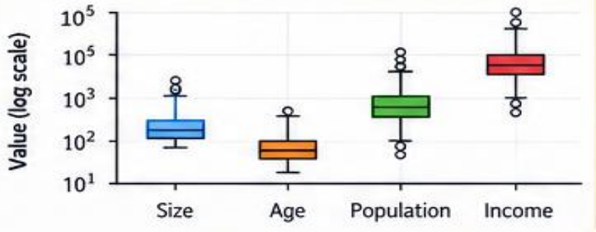
Missing values (NaN) · Inconsistent format / categories · Outliers (unrealistic values) · Mixed data types · Duplicate rows · Inconsistent text (spelling / case) · Logical errors (inconsistent values)

Index	House_ID	Size (m²)	Bedrooms	Age (years)	Location	Price ($1000)	Garage	Population	Income	Description	Built_Year
1	H001	120	3	10	Cairo	250	Yes	3000	60000	Nice house	2015
2	H002	85	2	15	cairo	180	no	2500	58000	nice House	2008
3	H003	NaN	3	5	Giza	200	Yes	2800	62000	Spacious	2020
4	H004	450	10	2	Alex	5000	Yes	10000	200000	Luxury villa	2019
5	H005	60	1	100	Alexandria	90	No	1200	30000	Old house	1920
6	H006	75	2	20	Giza	160	1	2500	55000	Good	2005
7	H007	200	4	8	cairo	300	Yes	5000	80000	New	2018
8	H008	90	2	NaN	Giza	NaN	No	2500	57000	good	2010
9	H009	120	3	10	Cairo	250	Yes	3000	60000	Nice house	2015
10	H010	5000	6	3	Aswan	750	Yes	80000	150000	Amazing view	2022
11	H011	70	two	12	Giza	140	No	2600	50000	Needs repair	2011
12	H012	110	3	-5	Cairo	220	Yes	3200	-1000	Good	2012
13	H013	95	2	18	Alexandria	abc	?	2700	52000	nice house	2016
14	H014	80	3	25	Alexandria	180	No	NaN	48000	Old	2014
15	H015	120	3	10	Cairo	250	Yes	3000	60000	Nice House	1900

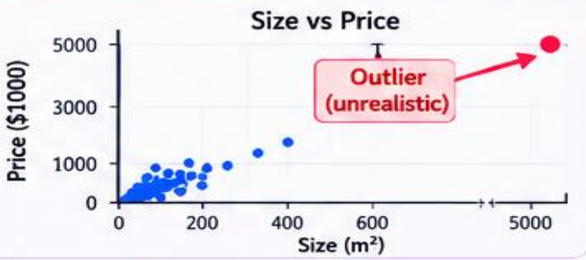
1. Missing Values



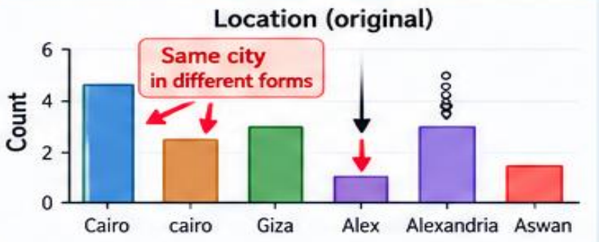
2. Different Scales



3. Outliers



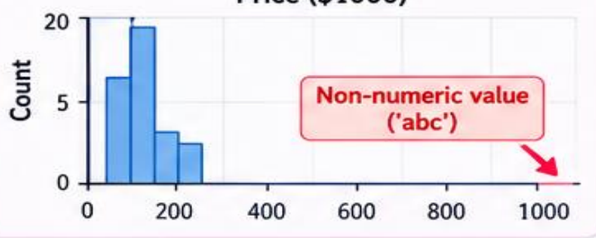
4. Inconsistent Categories



5. Mixed Data Types



6. Non-numeric Values in Numeric Column



7. Duplicate Rows

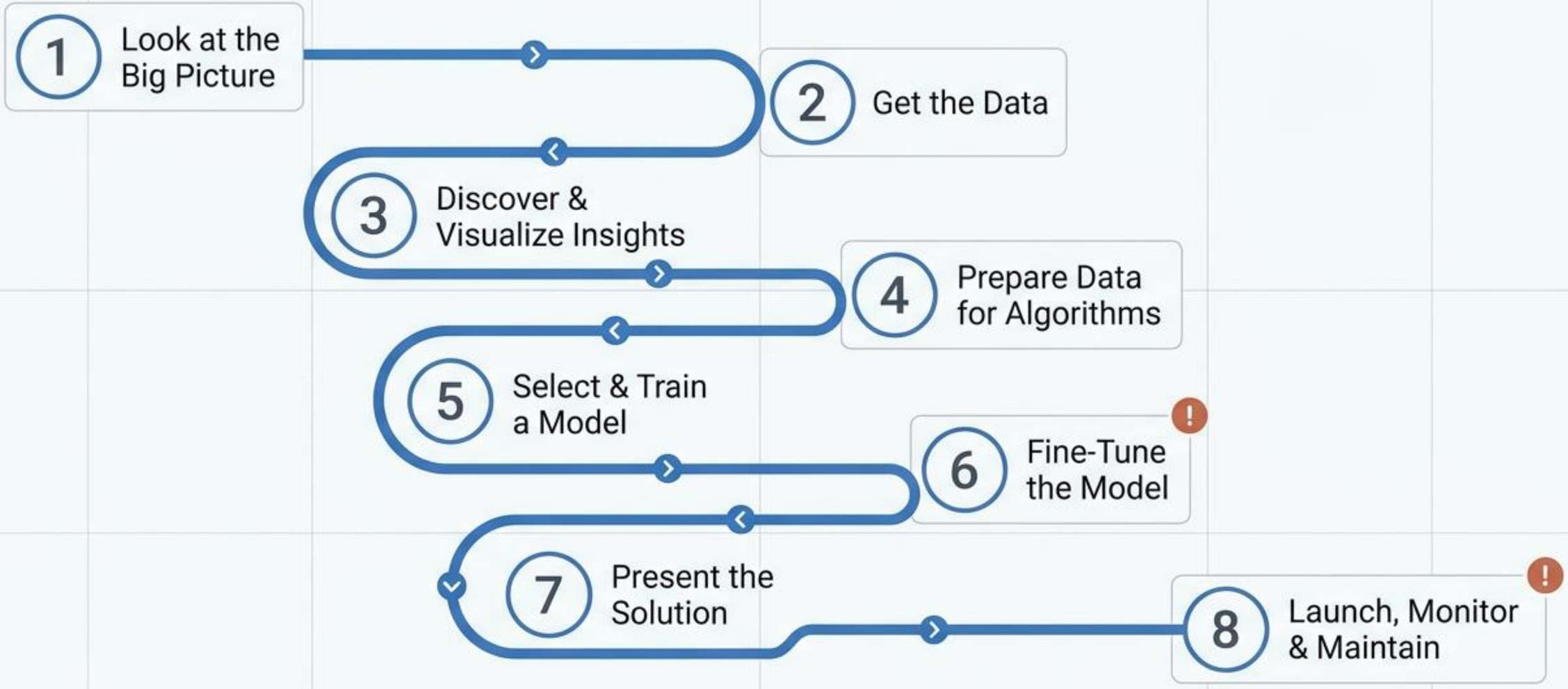
	House_ID	Size	Bedrooms	Age	Price
1	H001	120	3	10	250
9	H009	120	3	10	250
15	H015	120	3	10	250

Duplicate rows (same values)

8. Inconsistent / Invalid Values

Examples	Problem
Age = -5	Invalid (negative age)
Income = -1000	Invalid (negative income)
Built_Year = 1900	Unrealistic / inconsistent
Bedrooms = 10	Unrealistic (outlier)
Price = 5000	Unrealistic (outlier)
Size = 5000	Unrealistic (outlier)

The 8-Stage Engineering Lifecycle



Gemini Notebook

Today’s Roadmap

A complete ML workflow, grounded in the California housing example.

1 **Big Picture** → 2 **Get Data** → 3 **Explore** → 4 **Prepare** → 5 **Train** → 6 **Launch**

Core idea

An ML project is not just “train a model.” The value comes from the full system: problem framing, trustworthy data, careful evaluation, and continuous monitoring.

ONE WORKFLOW • MANY DECISIONS

Working with Real Data

Where can a real ML project begin?



01 Popular repositories

UC Irvine Machine Learning Repository
A large collection of real-world ML datasets.

Kaggle datasets
Wide variety of datasets, competitions, and notebooks.

Amazon AWS datasets
Open datasets for machine learning and data analytics.

02 Meta portals

Data Portals
A directory of open data repositories.

OpenDataMonitor
Search and explore open data from around the world.

Quandl
A platform providing economic, financial, and alternative datasets.

03 Other discovery sources

Wikipedia's ML dataset list
A comprehensive list of popular datasets.

Quora
Discussions and recommendations for useful datasets.

The datasets subreddit
A community sharing and discussing datasets.

Teaching example | **California Housing Prices Dataset** (StatLib Repository)

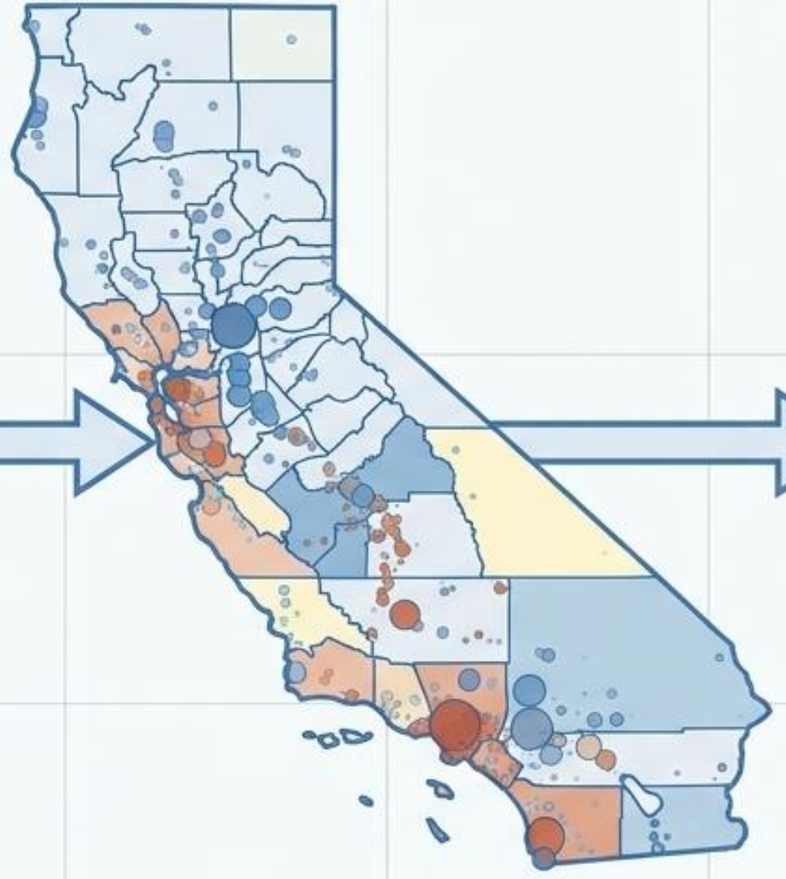


- **UCI Machine Learning Repository:** Browse UCI datasets
- **Kaggle Datasets:** Browse Kaggle datasets
- **Amazon AWS Open Data:** AWS Registry of Open Data
- **Data Portals:** Data Portals
- **OpenDataMonitor:** OpenDataMonitor
- **Quandl:** Quandl is now **Nasdaq Data Link**. Nasdaq Data Link
- **Wikipedia ML Dataset List:** List of datasets for machine-learning research
- **Quora:** Quora
- **Datasets subreddit:** r/datasets

Framing the Business Objective

Current State (Manual & Costly)

- District housing prices are currently estimated manually by experts.
- Highly costly and time-consuming.
- Estimates are frequently off by more than 20%.



The Objective (Automated Pipeline)

- Use California census data (population, median income) to build a predictive model.
- Target: Predict the median housing price for each district.

01

Look at the Big Picture

Before touching code, define what success means and how the model will be used.

Start with the Business Objective

The model exists to support a decision—not to maximize a metric in isolation.

Ask first

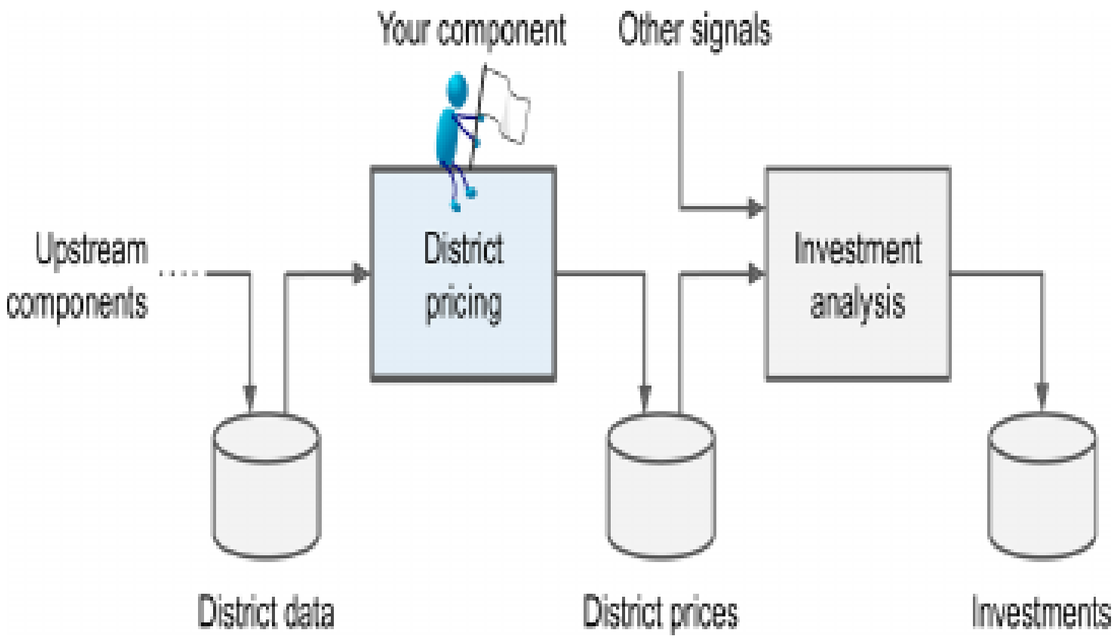
How will the company use and benefit from this model?

The objective determines…

- **Problem framing** — What exactly are we predicting?
- **Algorithm choice** — Which family of models fits?
- **Performance measure** — How will “good” be measured?
- **Tuning effort** — How much optimization is justified?

A Machine Learning Pipeline for Real Estate Investments

The pricing model is one component inside a larger decision system.



Pipeline

A sequence of data-processing components.

Why common in ML?

- Large volumes of data
- Many transformations
- Multiple downstream consumers

MACHINE LEARNING • END-TO-END PROJECT

Why Pipelines Matter?

They make complex ML systems **easier to build, scale, and recover.**



Asynchronous



- Components can run independently.
- Allows parallel execution.

Example

Data preprocessing can run while model training is running.

Data hand-off



- Each stage processes data and writes results.
- Clear input/output between stages.

Example

Cleaned data is saved and used by the next stage (model training).

Self-contained



- Teams can own separate components.
- Easy to develop, update and test.

Example

One team works on data preprocessing, another on model training.

Resilient



- If one stage fails, it can be rerun without starting from scratch.
- Helps in debugging and recovery.

Example

If model training fails, we can rerun only this stage using the last valid preprocessed data.

Understandable



- Clear flow of steps, inputs, and outputs.
- **Makes the system easier to reason about and maintain.**

Example

We can easily see which data and code are used at each stage.

Example: California Housing Prices Project Pipeline

1 Get the Data



California housing dataset (e.g., from StatLib)

2 Preprocess the Data



- Handle missing values
- Feature scaling
- Encode categorical features
- Split into train/test

3 Train the Model



- E.g., Linear Regression
- Train on training data
- Validate using cross-validation

4 Evaluate



- Metrics: RMSE, MAE
- Analyze errors
- Check feature importance

5 Deploy & Maintain



- Deploy the model
- Monitor performance
- Retrain with new data when needed

Benchmark the Current Solution

The existing process gives you a baseline and clues about what to improve.

CURRENT

Experts manually estimate district housing prices.

Costly • Time-consuming • >20% error

WHAT WE GAIN

- A reference for model performance
- Insights into how the problem is currently solved
- A concrete reason to automate

A new model should beat the baseline in a way that matters to the business.

Start Designing the ML System

Three decisions classify the learning setup.

01 **Learning type** — Supervised • Unsupervised • Reinforcement

02 **Task type** — Classification • Regression • Other

03 **Update strategy** — Batch learning • Online learning

Classifying the Housing Problem

Apply the design questions to the current example.

Supervised learning

Training examples include known target values.

Regression

Predict a continuous numeric value: house price.

Multiple regression

Several features contribute to one prediction.

Univariate output

Only one target value is predicted per district.

Batch learning

No continuous stream; data fits in memory.

Root Mean Square Error (RMSE)

Measures the average size of the prediction errors (in the same unit as the target).
For regression, RMSE is commonly used because large errors count more (they are squared).

Formula:

$$\text{RMSE}(X,h) = \sqrt{\frac{1}{m}\sum_{i=1}^{m}\left(h(x^{(i)}) - y^{(i)}\right)^2}$$

- m : number of instances (samples)
- $x^{(i)}$: feature vector of the i-th instance
- $y^{(i)}$: true label (target value)
- $h(x^{(i)})$: predicted value by the model
- X : feature matrix (all feature vectors)
- h : prediction function (model)

1 Input Data (X) and Target (y)

House	X (Features) Size (m^2)	X (Features) Bedrooms	y (Target) Price (1000$)
1	50	1	100
2	80	2	150
3	120	3	200

X: feature matrix (every row is $x^{(i)}$)

y: true target values (labels)

2 Model Predictions $h(x^{(i)})$

A trained model (h) predicts the house price for each house.



Model h(x)

Predicted Price $h(x^{(i)})$ (1000$)
110
140
160

3 Compute Errors

House	True Price $y^{(i)}$	Predicted $h(x^{(i)})$	Error $e^{(i)} = h(x^{(i)}) - y^{(i)}$
1	100	110	10
2	150	140	−10
3	200	160	−40

4 Square the Errors

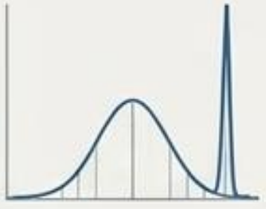
House	Error $e^{(i)}$	$(e^{(i)})^2$
1	10	100
2	−10	100
3	−40	1600

5 Calculate RMSE

$$\text{RMSE} = \sqrt{\frac{1}{m}\sum_{i=1}^{m}(e^{(i)})^2} \rightarrow = \sqrt{\frac{1}{3}(100 + 100 + 1600)} \rightarrow = \sqrt{\frac{1800}{3}} \rightarrow = \sqrt{600} \rightarrow \approx 24.49$$

The 40-unit miss (house 3) has a strong effect because errors are squared ($40^2 = 1600$), which is much larger than $10^2 = 100$.

Selecting the Engine: Evaluation Metrics

	Root Mean Square Error (RMSE)	Mean Absolute Error (MAE)
Mathematical Norm	l2 norm	l1 norm
Nickname	Euclidean Norm	Manhattan Norm
Sensitivity	Higher weight for large errors (Highly sensitive to outliers) 	Averages absolute deviations (Robust to outliers) 
Best Use Case	Standard multiple regression	Districts with many housing price outliers

RMSE vs. MAE

Outliers change which error measure may be more appropriate.

RMSE

$$\sqrt{\left[(1/m)\ \Sigma(\text{error})^2\right]}$$

More sensitive to large errors

Euclidean / L2 norm

MAE

$$(1/m)\ \Sigma|\text{error}|$$

More robust to outliers

Manhattan / L1 norm

Key idea: the higher the norm index, the more large values dominate.

Check the Assumptions

A small assumption check can prevent a large downstream failure.

LIST → VERIFY → CATCH ISSUES EARLY → SAVE REWORK

Example: if downstream users need price categories rather than exact prices, the problem may need to be reframed as classification.

Predict the exact house price → Regression

Income + Location + Rooms → Model → $350,000

Low Price / Medium Price / High Price

Income + Location + Rooms → Model → High Price

02

Get the Data

Set up the environment, inspect the dataset, and create a trustworthy test split.

Set Up the Workspace

A reproducible environment is the first technical step.

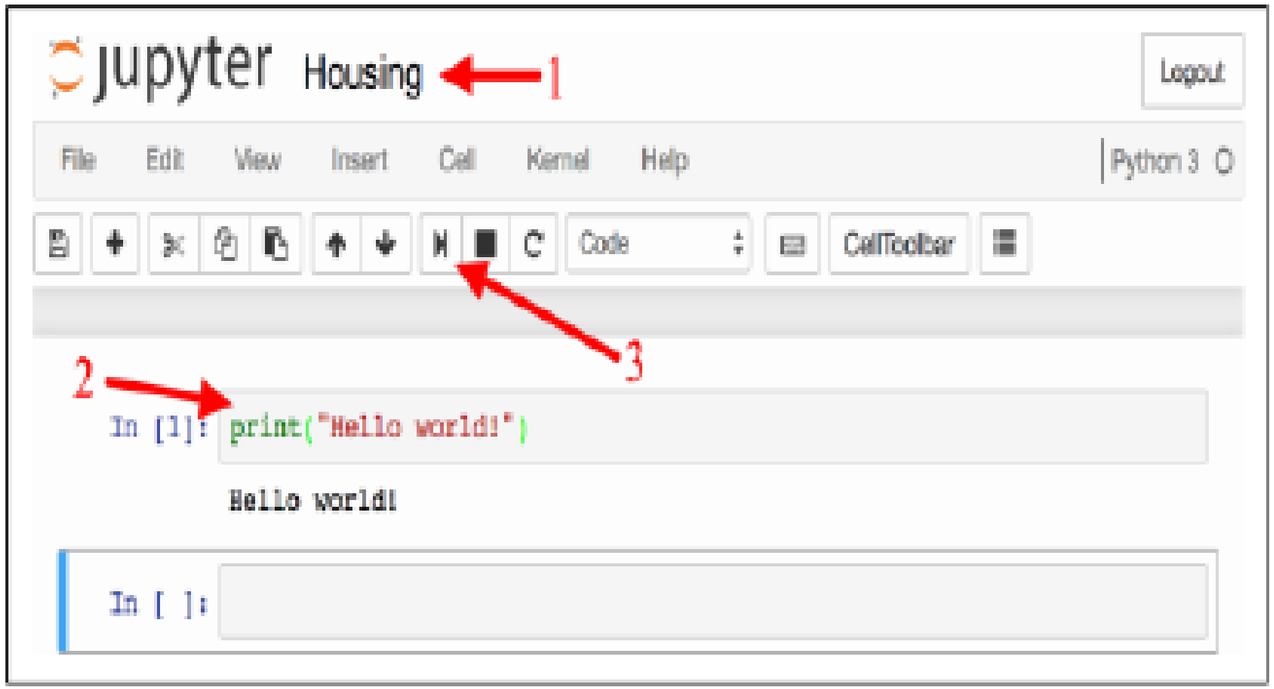
1. **Install Python**
2. **Create workspace**
3. **Install modules**
4. **Start Jupyter**

Jupyter server **http://localhost:8888/**

Keep code + datasets organized in one project workspace.

Hello World in a Python Notebook

Use notebooks to run code incrementally and inspect outputs.



Notebook loop

1. Title
2. Write
3. Inspect

A fast feedback cycle is ideal for exploratory ML work.

Download and Locate the Data

Real datasets often arrive from databases or multiple files—not one neat CSV.

Typical environment

- Relational databases
- Other common data stores
- Multiple tables / documents / files

Housing example

housing.tgz contains

housing.csv

comma-separated values with the full dataset

Take a Quick Look at the Data Structure

Start by understanding rows, columns, types, and obvious missing values.

```
In [5]: housing = load_housing_data()
        housing.head()
```

Out[5]:

	longitude	latitude	housing_median_age	total_rooms	total_bedrooms	populatio
0	-122.23	37.88	41.0	880.0	129.0	322.0
1	-122.22	37.86	21.0	7099.0	1106.0	2401.0
2	-122.24	37.85	52.0	1467.0	190.0	496.0
3	-122.25	37.85	52.0	1274.0	235.0	558.0
4	-122.25	37.85	52.0	1627.0	280.0	565.0

10 attributes

- longitude / latitude
- housing median age
- total rooms / bedrooms
- population / households
- median income
- median house value
- ocean proximity

Inspect Types and Summary Statistics

Two quick commands reveal different aspects of the dataset.

```
6]: housing.info()
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 20640 entries, 0 to 20639
Data columns (total 10 columns):
longitude             20640 non-null float64
latitude              20640 non-null float64
housing_median_age    20640 non-null float64
total_rooms           20640 non-null float64
total_bedrooms        20433 non-null float64
population            20640 non-null float64
households            20640 non-null float64
median_income         20640 non-null float64
median_house_value    20640 non-null float64
ocean_proximity       20640 non-null object
dtypes: float64(9), object(1)
memory usage: 1.6+ MB
```

housing.info()

```
3]: housing.describe()
```

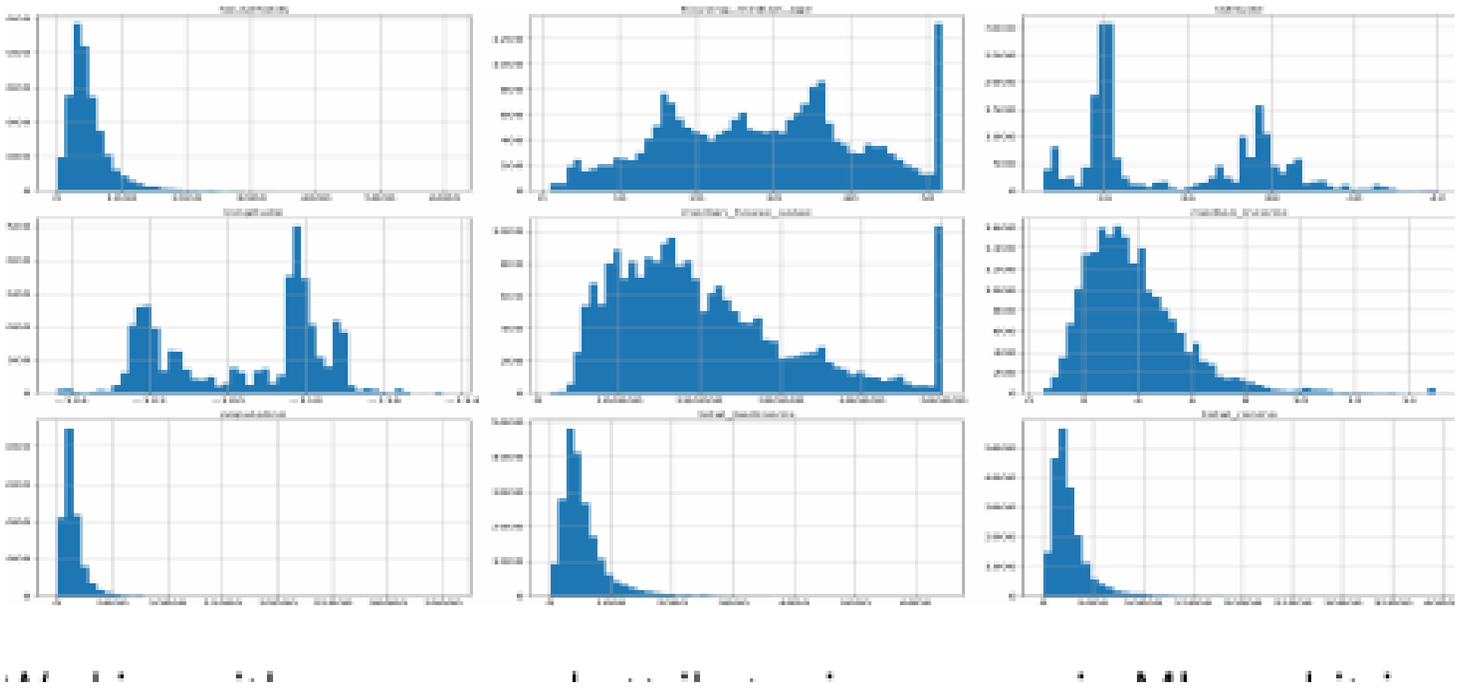
	longitude	latitude	housing_median_age	total_rooms	total_bed
count	20640.000000	20640.000000	20640.000000	20640.000000	20433.00
mean	-119.569704	35.631861	28.639486	2635.763081	537.8705
std	2.003532	2.135952	12.585558	2181.615252	421.3850
min	-124.350000	32.540000	1.000000	2.000000	1.000000
25%	-121.800000	33.930000	18.000000	1447.750000	296.0000
50%	-118.490000	34.260000	29.000000	2127.000000	435.0000
75%	-118.010000	37.710000	37.000000	3148.000000	647.0000
max	-114.310000	41.950000	52.000000	39320.000000	6445.000

housing.describe()

All attributes are numerical except ocean_proximity.

Histograms Reveal the Shape of the Data

Distribution plots help spot skew, caps, unusual scales, and preprocessing artifacts.



Remember

Preprocessed attributes are common—and not necessarily a problem.

But you should understand how each value was computed.

Training Data

Used to teach the model.



Test Data

Used to evaluate the model.
These examples are not seen during training.



1 Train the Model

The model learns from the training data (with known prices).



2 Learn Patterns

The model finds the relationship between features (size, location, etc.) and the price.



3 Test on New Data

We give the model new houses (it has never seen them before).



4 Evaluate Performance

Compare the predicted prices with the true prices (which are available only for the test set).



Create a Test Set Before Exploring

Protect the final evaluation from accidental information leakage.

Typical split



Scikit-Learn: train_test_split()

- random_state → reproducible seed
- Can split multiple aligned datasets on identical indices

Why set it aside?

The test set should simulate unseen data.

Do not tune decisions based on it.

Avoid Sampling Bias

A representative test set matters more than a merely random one.

	Overall	Stratified	Random	Rand. %error	Strat. %erro
1	0.039826	0.039729	0.040213	0.973236	-0.243309
2	0.318847	0.318798	0.324370	1.732260	-0.015195
3	0.350581	0.350533	0.358527	2.266446	-0.013820
4	0.176308	0.176357	0.167393	-5.056334	0.027480
5	0.114438	0.114583	0.109496	-4.318374	0.127011

Stratified sampling

Preserve the proportions of important groups—in this example, income categories.

Goal

Make the test set representative of the full dataset.

03

Discover & Visualize

Explore only the training data, search for patterns, and engineer better signals.

Explore the Training Set—Not the Test Set

Exploration is where you form hypotheses about the data.

1

Put the test set aside

2

Explore only the training set

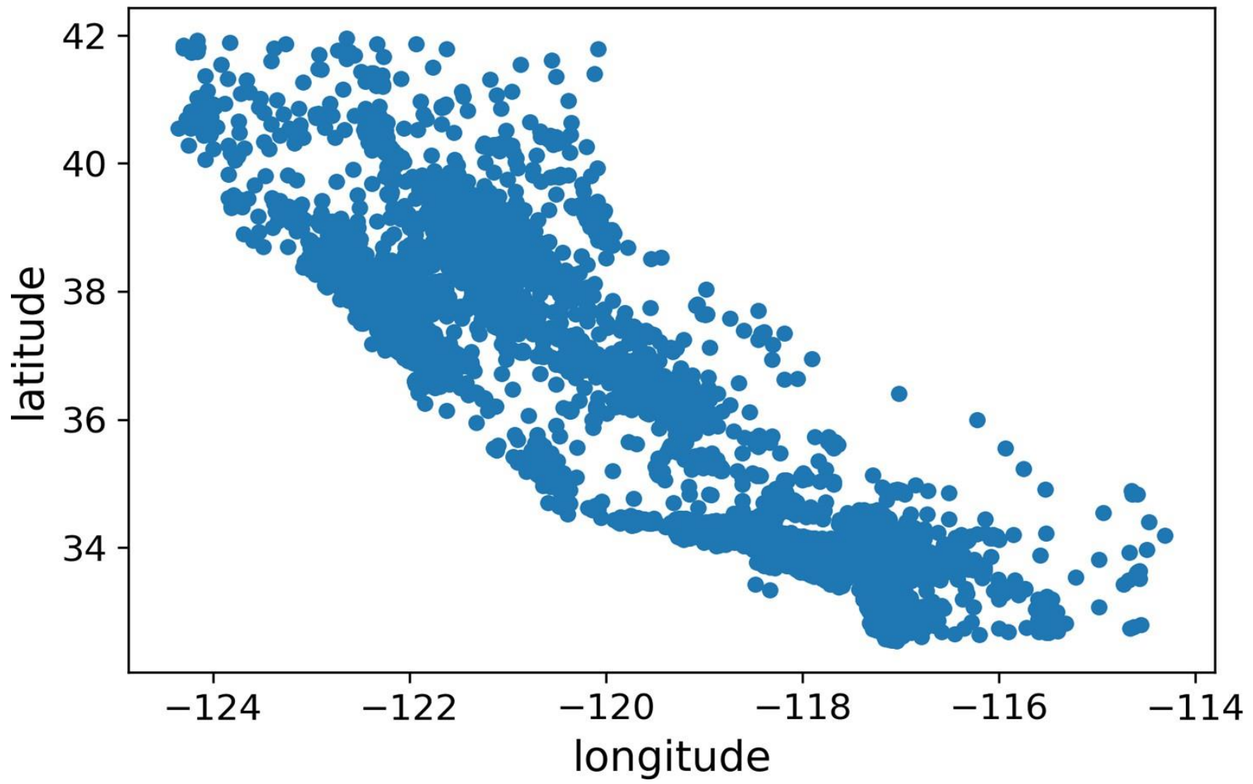
3

If very large, sample an exploration set

```
housing = strat_train_set.copy()
```

Visualizing Geographical Data

Longitude and latitude immediately reveal the shape of California.



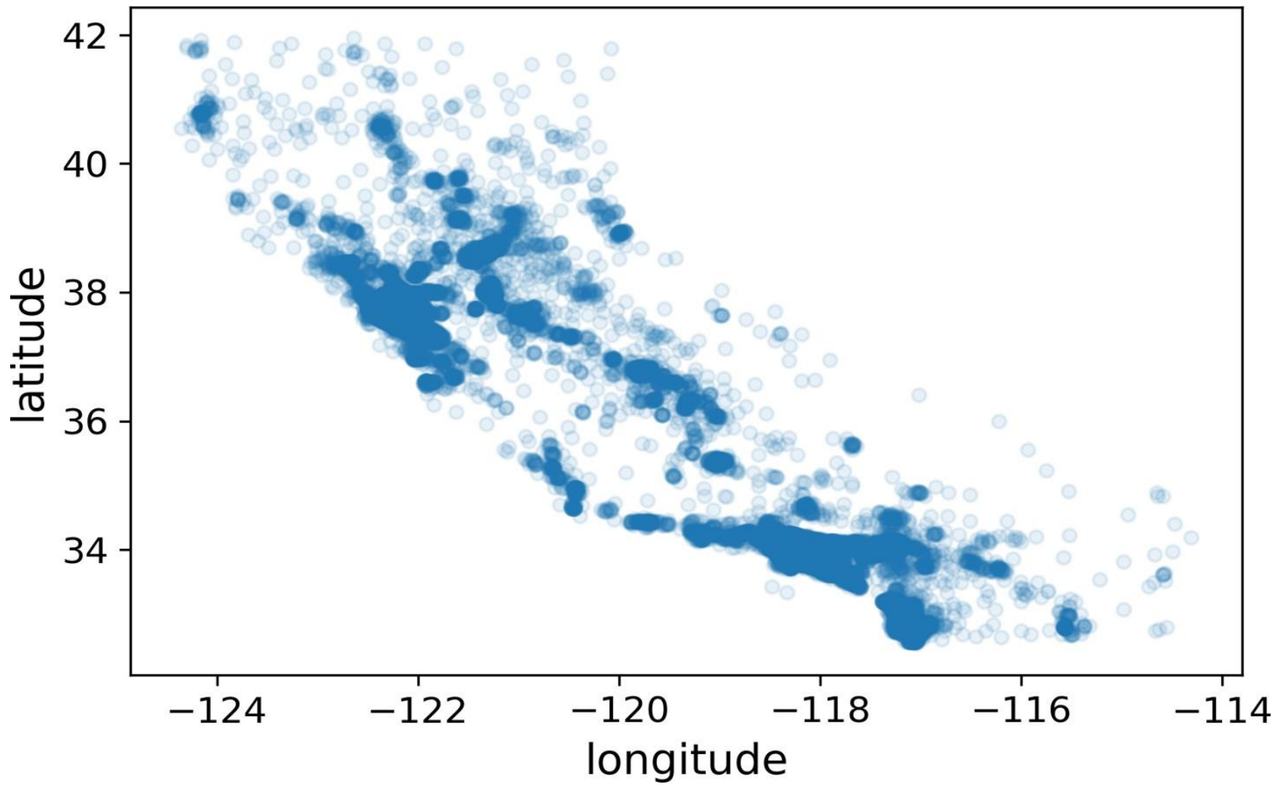
Code

```
housing.plot(
  kind="scatter",
  x="longitude",
  y="latitude"
)
```

First visual → location pattern

Transparency Highlights High-Density Areas

A small visualization change can reveal structure hidden by overlapping points.



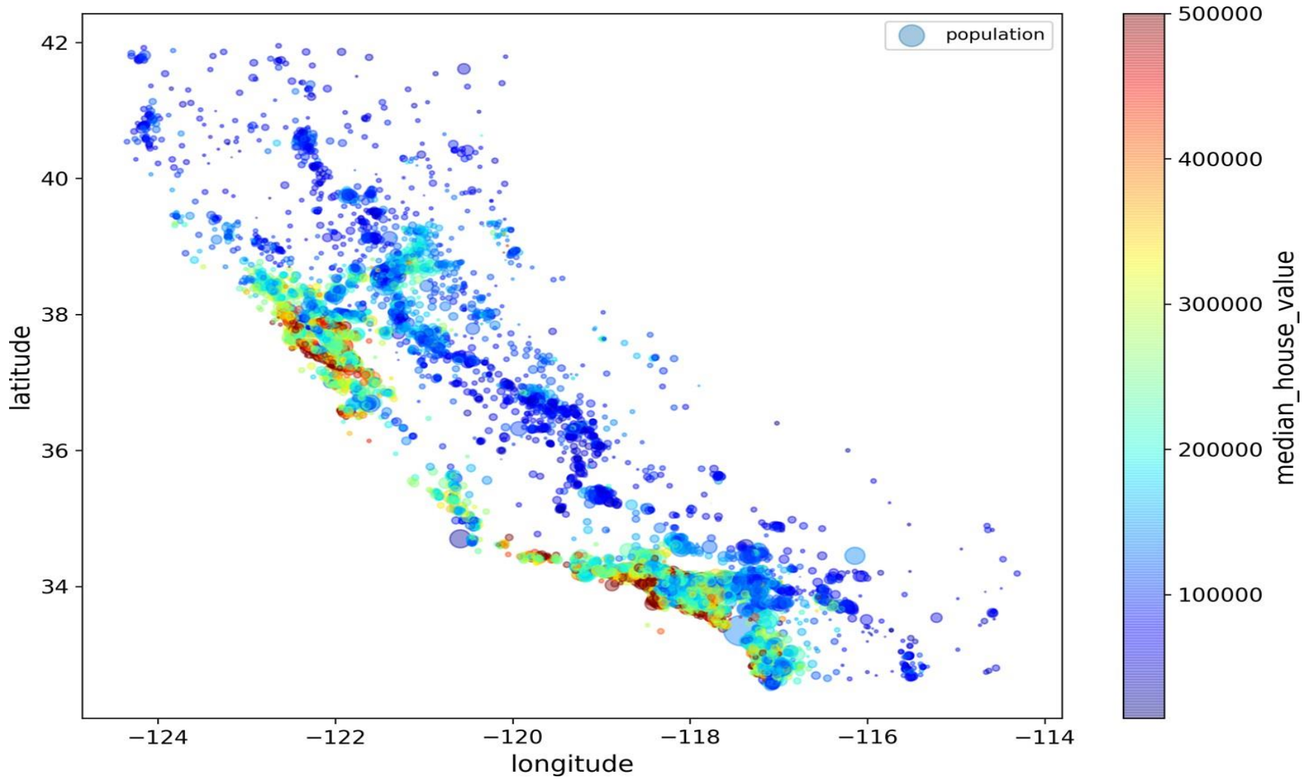
Add alpha = 0.1

Overlapping points become darker, making dense regions easier to see.

VISUAL ENCODING

Encode More Information in One Plot

Color → house value • Circle size → population • Position → geography



Read the visual

- Red = expensive
- Blue = cheap
- Larger circles = larger population

Multiple encodings help reveal geographic and economic patterns together.

housing.plot(kind=”scatter”, x=”longitude”, y=”latitude”, alpha=0.4, s=housing[”population”]/100, label=”population”, figsize=(10,7), c=”median house value”, cmap=plt.get cmap(”jet”), colorbar=True)

Pearson Correlation Coefficient (r)

Measures the strength and direction of the **linear relationship** between two numerical variables.

Range: $-1 \leq r \leq 1$

- r close to 1 → **strong positive** correlation
- r close to −1 → **strong negative** correlation
- r close to 0 → **no linear** correlation

Formula:

$$r = \frac{\sum_{i=1}^{n}(x_i - \bar{x})(y_i - \bar{y})}{\sqrt{\sum_{i=1}^{n}(x_i - \bar{x})^2}\sqrt{\sum_{i=1}^{n}(y_i - \bar{y})^2}}$$

Example Dataset (Houses)

House	Size (m²)	Bedrooms	Age (years)	Price ($1000)
1	50	1	30	100
2	80	2	20	150
3	120	3	10	200
4	150	3	5	250
5	200	4	2	400

Compute using Python

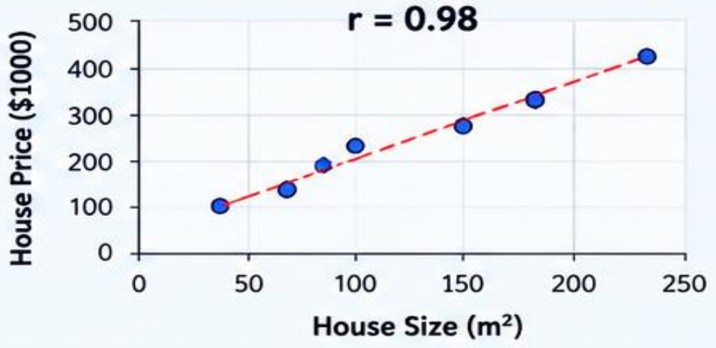
```
import pandas as pd
corr = df.corr()
print(corr)
```

Correlation Matrix (example output)

	Size	Bedrooms	Age	Price
Size	1.00	0.95	-0.80	0.98
Bedrooms	0.95	1.00	-0.70	0.94
Age	-0.80	-0.70	1.00	-0.78
Price	0.98	0.94	-0.78	1.00

Strong Positive Correlation (r ≈ 1)

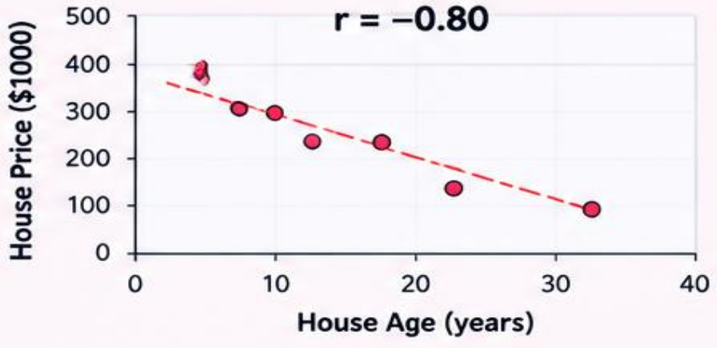
As one variable increases, the other variable also increases.



Larger houses tend to have higher prices.

Strong Negative Correlation (r ≈ −1)

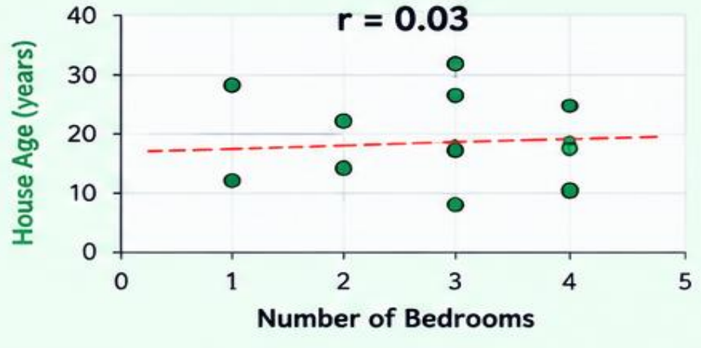
As one variable increases, the other variable decreases.



Older houses tend to have lower prices.

No Linear Correlation (r ≈ 0)

There is no clear linear relationship between the variables.



Number of bedrooms is not linearly related to the age of the house.

1) Example Dataset (Houses)

House	Size (m²) X_1	Bedrooms X_2	Age (years) X_3	Price ($1000) Y
1	50	1	30	100
2	80	2	20	150
3	120	3	10	200
4	150	3	5	250
5	200	4	2	400

2) Pearson Correlation Formula

$$r = \frac{\sum_{i=1}^{n}(x_i - \bar{x})(y_i - \bar{y})}{\sqrt{\sum_{i=1}^{n}(x_i - \bar{x})^2 \sum_{i=1}^{n}(y_i - \bar{y})^2}}$$

- r ranges from -1 to 1
- $r \approx 1$ → strong positive
- $r \approx -1$ → strong negative
- $r \approx 0$ → no linear correlation

3) Compute the Means

$$\bar{x}_{size} = \frac{50+80+120+150+200}{5} = 120$$

$$\bar{x}_{bed} = \frac{1+2+3+3+4}{5} = 2.6$$

$$\bar{x}_{age} = \frac{30+20+10+5+2}{5} = 13.4$$

$$\bar{y}_{price} = \frac{100+150+200+250+400}{5} = 220$$

4) Example 1: Correlation between Size and Price

House	Size x_i	Price y_i	$x_i - \bar{x}$ $(x_i - 120)$	$y_i - \bar{y}$ $(y_i - 220)$	$(x_i - \bar{x})(y_i - \bar{y})$	$(x_i - \bar{x})^2$	$(y_i - \bar{y})^2$
1	50	100	−70	−120	8,400	4,900	14,400
2	80	150	−40	−70	2,800	1,600	4,900
3	120	200	0	−20	0	0	400
4	150	250	30	30	900	900	900
5	200	400	80	180	14,400	6,400	32,400
Sum					**26,500**	**13,800**	**53,000**

$$r_{size,price} = \frac{26,500}{\sqrt{13,800 \times 53,000}} = \frac{26,500}{27,051.8} \approx \mathbf{0.98}$$

$r \approx 0.98$ → strong positive correlation

5) Example 2: Correlation between Age and Price

House	Age x_i	Price y_i	$x_i - \bar{x}$ $(x_i - 13.4)$	$y_i - \bar{y}$ $(y_i - 220)$	$(x_i - \bar{x})(y_i - \bar{y})$
1	30	100	16.6	−120	−1,992
2	20	150	6.6	−70	−462
3	10	200	−3.4	−20	68
4	5	250	−8.4	30	−252
5	2	400	−11.4	180	−2,052
Sum					**−4,690**

$$r_{age,price} = \frac{-4,690}{\sqrt{525.2 \times 53,000}} = \frac{-4,690}{5,274.9} \approx -0.89$$

$r \approx -0.89$ (≈ -0.78 in the figure) → strong negative

6) Example 3: Correlation between Bedrooms and Age

House	Bedrooms x_i	Age y_i	$x_i - \bar{x}$ $(x_i - 2.6)$	$y_i - \bar{y}$ $(y_i - 13.4)$	$(x_i - \bar{x})(y_i - \bar{y})$
1	1	30	−1.6	16.6	−26.56
2	2	20	−0.6	6.6	−3.96
3	3	10	0.4	−3.4	−1.36
4	3	5	0.4	−8.4	−3.36
5	4	2	1.4	−11.4	−15.96
Sum					**−51.20**

$$r_{bed,age} = \frac{-51.20}{\sqrt{2.80 \times 525.20}} = \frac{-51.20}{38.31} \approx -0.67$$

$r \approx -0.67$ (≈ -0.70 in the figure) → strong negative

7) Final Correlation Matrix (rounded)

	Size	Bedrooms	Age	Price
Size	1.00	0.95	−0.80	0.98
Bedrooms	0.95	1.00	−0.70	0.94
Age	−0.80	−0.70	1.00	−0.78
Price	0.98	0.94	−0.78	1.00

Experiment with Attribute Combinations

Raw totals may be less informative than ratios that match the real question.

rooms / household

total_rooms ÷ households

How spacious is a typical household?

bedrooms / rooms

total_bedrooms ÷ total_rooms

What fraction of rooms are bedrooms?

population / household

population ÷ households

How crowded is a household?

Feature engineering turns domain intuition into model-ready signals.

- Try out various attribute combinations.
- The **total number of rooms in a district** is **not very useful** if you don't know how many households there are. What you really want is the number of rooms per household.
- The **total number of bedrooms** by itself is not very useful: you probably want to compare it to the number of rooms.
- The population per household also seems like an interesting attribute combination to look at.

04

Prepare the Data

Clean missing values, encode categories, scale features, and make transformations reusable.

Write Reusable Data-Preparation Functions

Transformations should work the same way in experiments and in production.

Reproduce

Apply the same transformations to any dataset.

Reuse

Build a library for future projects.

Deploy

Transform new live data before prediction.

Compare

Try transformation combinations systematically.

Data Cleaning: Missing Values

Three basic strategies—plus a reusable Scikit-Learn transformer.

Drop rows

```
housing.dropna(subset=["total_bedrooms"])
```

Drop attribute

```
housing.drop("total_bedrooms", axis=1)
```

Impute

Fill with zero, mean, median, etc.

- Set the values to some value (zero, the mean, the median, etc.):
  median = housing[”total bedrooms”].median() housing[”total bedrooms”].fillna(median, inplace=True)
- Scikit-Learn provides a handy class to take care of missing values: SimpleImputer.

Scikit-Learn provides a dedicated class for missing-value imputation.

SimpleImputer

Handling Text and Categorical Attributes

Most ML algorithms need numeric inputs, so categories must be encoded.

ocean_proximity

- OCEAN
- INLAND
- ISLAND
- NEAR BAY
- NEAR OCEAN

OrdinalEncoder

Category → integer

Use when an ordered numeric representation is appropriate.

OneHotEncoder

INLAND → 0 1 0 0 0

One attribute is 1 (hot); the others are 0 (cold).

Feature Scaling

Same data, two different methods

Original data: 10 | 20 | 30

Min–Max Scaling

Scale the observed values to [0, 1]

$$x' = \frac{x - \min}{\max - \min}$$

min = 10 max = 30

Example: $x = 30$

$$x' = \frac{30 - 10}{30 - 10} = 1$$

Original	Scaled
10	0
20	0.5
30	1

Minimum → 0 Maximum → 1

Standardization

Express distance from the mean in standard deviations

$$z = \frac{x - \mu}{\sigma}$$

$\mu = 20$ $\sigma \approx 8.165$

Example: $x = 30$

$$z = \frac{30 - 20}{8.165} \approx 1.225$$

Original	z-score
10	−1.225
20	0
30	+1.225

Mean → 0 Standard deviation → 1

Where does 8.165 come from?

σ is the standard deviation, calculated from the data.

1 Find the mean

$$\mu = \frac{10 + 20 + 30}{3} = 20$$

2 Subtract the mean, then square

Value x	$x - \mu$	$(x - \mu)^2$
10	−10	100
20	0	0
30	+10	100

3 Average the squared differences

$$\text{Variance} = \frac{100 + 0 + 100}{3} \approx 66.667$$

Divide by 3 because there are 3 values.

4 Take the square root

$$\sigma = \sqrt{\frac{200}{3}} \approx 8.165$$

Use this standard deviation in the standardization formula:

$$z = \frac{30 - 20}{8.165} \approx 1.225$$

8.165 ← standard deviation

$z = 1.225$ means: 30 is 1.225 standard deviations above the mean.

This example uses population standard deviation (divide by n). Standardization is not limited to [0, 1].

Worked Example: Why Scaling Changes the Geometry

A feature measured in thousands can dominate one measured in tens.

Before scaling

Median income = 4
Population = 2,000

Distance-based algorithms “see” population as much larger numerically.

After scaling

Both features contribute on a comparable numerical scale.

Standardization is much less affected by outliers than min-max scaling.

05

Select, Train & Fine-Tune

Compare models, validate robustly, and optimize hyperparameters without touching the test set.

Training and Evaluating on the Training Set

Two models illustrate the classic underfitting ↔ overfitting tension.

Linear Regression

Simple model

→ may underfit the training data

Too simple to capture important structure.

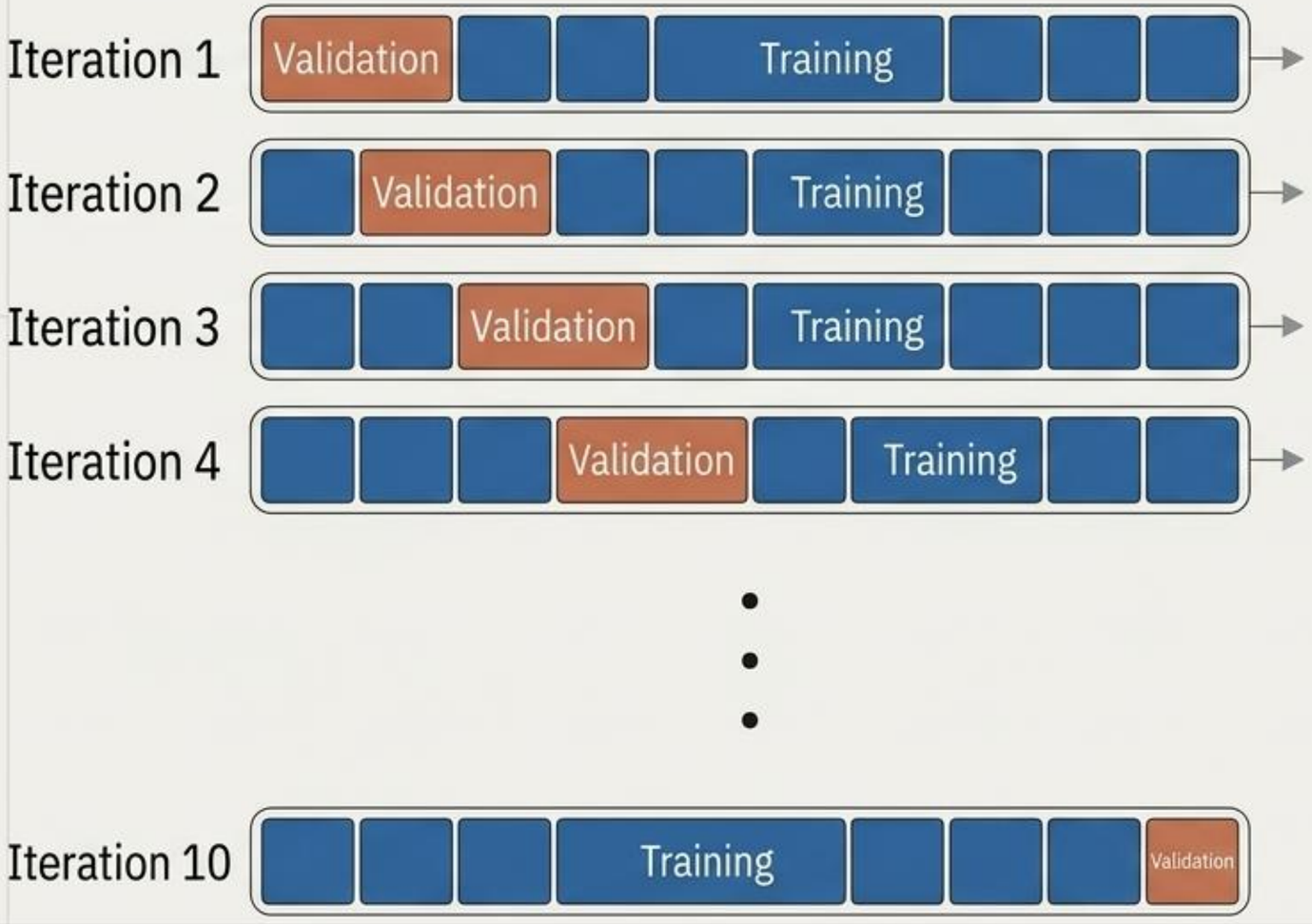
DecisionTreeRegressor

Powerful nonlinear model

→ may overfit the training data

Can memorize details that do not generalize.

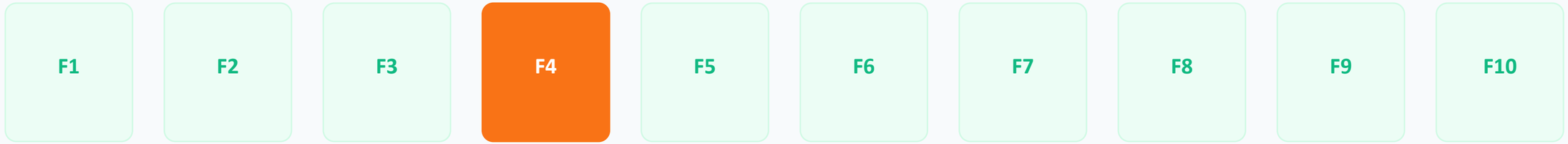
Robust Training: K-Fold Cross-Validation



- Scikit-Learn's K-fold cross-validation prevents overfitting by splitting the training set into 10 distinct subsets (folds).
- The model trains on 9 folds and evaluates on 1. This process is repeated 10 times, cycling the validation fold.
- Result: A highly accurate, robust performance estimate.

Better Evaluation with K-Fold Cross-Validation

Use multiple validation folds instead of trusting one split.



Train on 9 folds • Validate on 1 • Rotate the validation fold

Cross-validation gives a more reliable estimate of how the model generalizes.

Random Forest: Ensemble Learning in Action

Many diverse decision trees can outperform one individual tree.



RandomForestRegressor trains many trees on random feature subsets, then averages their predictions.

ENSEMBLE LEARNING

Fine-Tune the Model

Grid search and randomized search explore hyperparameter settings differently.

GridSearchCV

Try all specified combinations

✓ systematic
✓ cross-validation evaluation
△ expensive when the grid grows

RandomizedSearchCV

Sample a fixed number of random combinations

✓ efficient for large spaces
✓ explores more values
△ not exhaustive

From Fine-Tuning to Final Evaluation

Combine strong models, inspect errors, then evaluate once on the test set.

1 **Best models**

2 **Ensemble**

3 **Analyze errors**

4 **Final test**

Ensemble methods often perform better than the best individual model.

Only after model selection is complete should the final system be evaluated on the untouched test set.

06

Launch, Monitor & Maintain

Production is the beginning of the model’s operational life—not the end of the project.

Get the Solution Ready for Production

A model becomes useful only when it can run reliably in a real environment.

Polish code

Make it maintainable and production-ready.

Documentation

Explain usage, assumptions, and interfaces.

Tests

Verify data and prediction behavior.

Deploy

Move the model into the production environment.

Monitoring Closes the Loop

Once deployed, watch both the system and the model’s real-world behavior.

1 **New data** → 2 **Prediction** → 3 **Monitor** → 4 **Maintain**

Why monitor? **Data changes, performance can degrade, and operational failures can appear after launch.**

End-to-End ML Project: The Big Picture

The lecture’s workflow in one final view.

1 **Frame** › 2 **Acquire** › 3 **Explore** › 4 **Prepare** › 5 **Model** › 6 **Operate**

1. **Protect the test set until the end.**
2. **Use metrics that match the problem and error costs.**
3. **Build reusable preprocessing pipelines.**
4. **Validate and tune before final testing.**
5. **Deployment requires monitoring and maintenance.**

Quick Concept Check

Use these questions to reinforce the most important decisions.

Why is housing price prediction a regression task?

Because the target is a continuous numeric value.

When might MAE be preferred over RMSE?

When outliers are common and large errors should not dominate as strongly.

Why stratify the test set?

To preserve important group proportions and reduce sampling bias.

Why use cross-validation?

To estimate generalization more reliably than a single validation split.